

React Native Technical Assignment

Mutual Fund Explorer

Objective

Build a React Native (Expo) application that lets a user explore Indian mutual funds, view a fund's NAV history, maintain a watchlist, and record simple holdings to see current value and returns.

Use the free public API at mfapi.in — it provides scheme search and full NAV history for Indian mutual funds, with no authentication required.

The goal of this assignment is to evaluate:

- Application structure and state management
- Form handling and input validation
- Third-party API integration and handling of real-world data
- Handling of loading, empty, error, and edge-case states
- Working correctly with dates and money
- List and rendering performance
- Engineering judgment and decision-making
- Use of AI-assisted development tools

Please focus on practical implementation and clear decision-making rather than over-engineering.

Expected Submission Timeline: 3 days

Technical Stack

Required:

- React Native using Expo (managed workflow)
- TypeScript
- A state management approach of your choice (Context, Redux, Zustand, etc.)

Preferred (optional):

- Local persistence so the watchlist and holdings survive app restarts
- A clean, reusable component structure

The application should run locally via Expo Go or a development build.

Functional Requirements

Your application should support a flow where:

- A user can search for mutual fund schemes by name
- A user can open a fund and view its details along with its NAV history in a readable form (chart and/or list)
- A user can add funds to a watchlist showing each fund's latest NAV and last-updated date
- A user can record a simple holding for a fund (units and purchase date), and the app computes the current value and overall return using NAV data
- A user can remove funds from the watchlist / holdings
- Values are presented with correct currency and number formatting

You are free to decide:

- App navigation and screen structure
- State management approach
- How to present NAV history (chart, list, summary statistics — your call)
- How to compute returns, and how to handle dates where NAV data is unavailable (weekends, holidays, gaps)
- Persistence approach (if any)
- How to handle API failures, slow responses, and incomplete data

Please document important architectural decisions.

Quality Expectations

The app should behave sensibly in real-world conditions, not just the happy path. Please consider:

- Loading, empty, and error states for every data-driven screen
- Graceful handling when the API is slow, unavailable, or returns incomplete data
- NAV history responses can be large (thousands of entries) — screens should remain smooth
- Correct handling of dates: purchase dates that fall on non-trading days, funds with short or gappy history
- Correct, readable formatting of NAVs, units, values, and returns (including negative returns)
- Input validation on the holding form (units must be a valid positive number; purchase date must be sensible)

Deliverables

Please submit:

- Git repository or ZIP
- Setup instructions (how to install and run via Expo)
- Any environment/config notes needed to run the app
- AI_CONTEXT.md (see below)

AI_CONTEXT.md

Please include a short note covering:

- AI tools used
- Where AI helped
- What required manual correction or redesign
- Key architectural decisions and trade-offs
- A specific example where you overrode or corrected the AI's suggestion, and why
- What you would improve with more time

We are more interested in how you think and direct AI tools than in whether the AI produced working code. Show us your reasoning.

Evaluation Criteria

We will evaluate:

- Code quality and readability
- Application and component structure
- State management approach

- API integration and handling of real-world data
- Correctness of returns computation and date handling
- Handling of loading, empty, error, and edge-case states
- List and rendering performance with large datasets
- Input validation and formatting
- Engineering judgment
- Ability to explain technical decisions

A polished, production-grade UI is not required, but a clean and thoughtful one is expected. Please focus on correctness, clarity, maintainability, and practical implementation.